Batch Code: ANP_D1838

Student ID: AF05202193

Student Name: S. Vaishnavi

LAB-09

Question:

1. Create a java program using compile time - polymorphism and explanation
2. Create a java program using run time - polymorphism and explanation
3. Difference between method overloading and Method Overriding
4. Difference between compile time polymorphism and run time polymorphism

---

1. COMPILE TIME – POLYMORPHISM

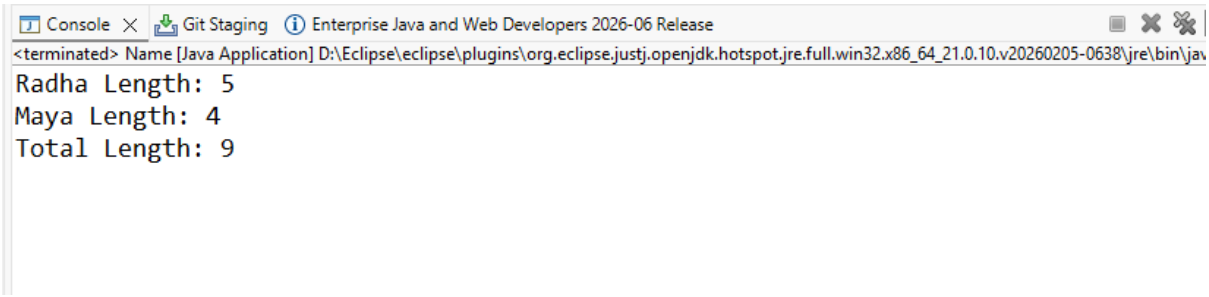
- Compile-time polymorphism is a type of polymorphism where the method to be executed is decided by the compiler during the compilation process. It is achieved using method overloading, where multiple methods have the same name but different parameters.
- If a class contains multiple add () methods with different parameter lists, the compiler chooses the correct method based on the arguments passed.

PROGRAM:

```
public class Name {
   int member(String name) {
      return name.length();
   }
   int member(String name1, String name2) {
```

```
        return name1.length() + name2.length();
    }
    public static void main(String[] args) {
        Name obj = new Name();
        System.out.println("Radha Length: " + obj.member("Radha"));
        System.out.println("Maya Length: " + obj.member("Maya"));
        System.out.println("Total Length: " + obj.member("Radha", "Maya"));
    }
}
```

OUTPUT:



EXPLANATION

- member("Radha") returns 5.
- member("Maya") returns 4.
- member("Radha", "Maya") returns 9.
- Same method name member() is used with different parameters.
- This is called Method Overloading.
- Method Overloading is an example of Compile-Time Polymorphism.

COMMENTS:

LINE OF CODE	EXPLANATION
public class Name	Creates a class named Name.
int member(String name)	Method with one String parameter.
return name.length();	Returns the length of the given name.
int member(String name1,String name2)	Overloaded method with two String parameters.
return name1.length() + name2.length();	Returns the total length of both names.
public static void main(String[] args)	Main method where program execution starts.
Name obj = new Name();	Creates an object of the Name class.
obj.member("Radha")	Calls the first method and returns 5.
obj.member("Maya")	Calls the first method and returns 4.
obj.member("Radha", "Maya")	Calls the overloaded method and returns 9.
System.out.println()	Prints the result on the console.

---

2. RUNTIME POLYMORPHISM

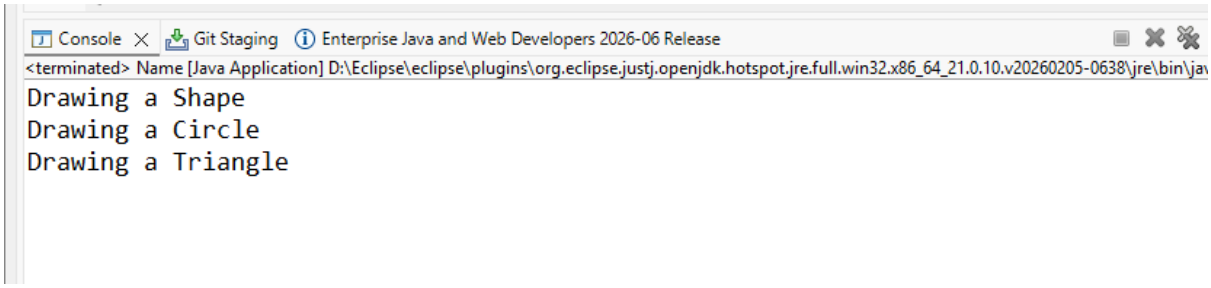
Run-Time Polymorphism is a type of polymorphism in which the method to be executed is determined during program execution (runtime). It is achieved through Method Overriding, where a child class provides its own implementation of a method already defined in the parent class.

PROGRAM:

```
class Shape {
    void draw() {
        System.out.println("Drawing a Shape");
    }
}
class Circle extends Shape {
    @Override
    void draw() {
        super.draw();
        System.out.println("Drawing a Circle");
    }
}
class Triangle extends Circle {
    @Override
    void draw() {
        System.out.println("Drawing a Triangle");
    }
}
public class Shapes {
    public static void main(String[] args) {
        Shape c = new Circle();
```

```
        Shape t = new Triangle();

        c.draw();

        t.draw();

    }

}
```

OUTPUT:



EXPLANATION

- Shape c = new Circle(); creates a Circle object using a Shape reference.
- Shape t = new Triangle(); creates a Triangle object using a Shape reference.
- c.draw(); calls the draw() method of the Circle class.
- super.draw(); inside Circle calls the draw() method of the Shape class first.
- t.draw(); calls the draw() method of the Triangle class.
- The same method name draw() is used in Shape, Circle, and Triangle classes.
- This is called Method Overriding.
- Method Overriding is an example of Run-Time Polymorphism.
- The JVM decides which draw() method to execute during runtime based on the object created.

COMMENTS:

CODE	EXPLANATION
class Shape	Creates the parent class.
void draw()	Method defined in the parent class.
class Circle extends Shape	Circle inherits properties of Shape.
@Override	Indicates that the method is overridden.
super.draw();	Calls the parent class (Shape) method.
System.out.println("Drawing a Circle");	Prints Circle message.
class Triangle extends Circle	Triangle inherits Circle class.
void draw()	Overrides the draw() method.
Shape c = new Circle();	Parent reference points to Circle object.
Shape t = new Triangle();	Parent reference points to Triangle object.
c.draw();	Executes Circle's draw() method at runtime.
t.draw();	Executes Triangle's draw() method at runtime.

---

3. METHOD OVERLOADING AND METHOD OVERRIDING

METHOD OVERLOADING	METHOD OVERRIDING
Same method name with different parameters	Same method name and same parameters
Occurs in the same class	Occurs between parent and child classes

Compile-time binding	Runtime binding
Inheritance is not required	Inheritance is required
Improves code readability	Provides specific implementation
Example: add(int,int) and add(int,int,int)	Example: Animal.sound() overridden by Dog.sound()

---

4. DIFFERENCE BETWEEN COMPILE-TIME POLYMORPHISM AND RUN-TIME POLYMORPHISM

COMPILE-TIME POLYMORPHISM	**RUN-TIME POLYMORPHISM**
Achieved through Method Overloading	Achieved through Method Overriding
Method call resolved during compilation	Method call resolved during execution
Faster execution	Slightly slower due to dynamic binding
Inheritance not required	Inheritance required
Also called Static Polymorphism	Also called Dynamic Polymorphism
Example: Overloaded member() methods	Example: Overridden draw() method

---